

152113017: NESNE TABANLI PROGRAMLAMA I
FİNAL SINAVI (11.01.2023)

Kitap ve notlar kapalı

Toplam: 100 puan, 4 Soru, Süre: 90 dakika

S.1	S.2	S.3	Toplam
20	25	55	100

Adı-Soyadı: CEVAPLAR

Numara: CEVAPLAR

Code.h

```
#include <iostream>
#include <string>
using namespace std;
class Code {
    int code;
public:
    Code(int c);
    bool compare(const Code &) const;
};
```

Code.cpp

```
#include "code.h"
Code::Code(int c) :code(c) {}

bool Code::compare(const Code &r) const
{
    return (code == r.code);
}
```

Account.h

```
#include "code.h"
class Account {
    Code code;
    const int number;
    double balance;
public:
    Account(int, double, Code &);
    bool operator==(const Account&);
    double getBalance() const;
    int getNumber() const;
    double withdraw(double);
};
```

Account.cpp

```
#include "Account.h"
Account::Account(int n, double b, Code &c)
    :number(n), balance(b),code(c) {}

double Account::getBalance() const
{
    return balance;
}

int Account::getNumber() const
{
    return number;
}

double Account::withdraw(double amount)
{
    if ((balance-amount) >= 0) {
        balance -= amount;
        return amount;
    }
    else
        return 0;
}

bool Account::operator==(const Account& account)
{
    return (code.compare(account.code));
}
```

S.1 (Template, 20p.)

(a) [12p] Code adlı sınıfı, bir şablon (template) sınıf olarak tekrar yazınız. Öyle ki, code adlı üye verisi int dışında da farklı tiplerde olabilsin (string, int, long, vb.).

Cevap:

Code.h

```
template <typename T>
class Code {
    T code;
public:
    Code(T);
    bool compare(Code<T> &) const;
};
```

Code.cpp

```
template <typename T>
Code<T>::Code(T c) :code(c) {}

template <typename T>
bool Code<T>::compare(const Code<T> &r)
const{
    return (code == r.code);
}
```

(b) [8p] Sorunun bir önceki şıkında, Code sınıfı template sınıf olarak kodlandı. Account sınıfında, code adlı üye int tipi değerle kullanmaya devam etmek istiyorsak, bu durumda Account sınıfını da değiştirmemiz gerekir mi? Evet ise, sadece değişiklik olan kodları yazınız. Hayır, ise, nedenini açıklayınız.

Cevap:

```
class Account {
    Code<int> code;
    ...
private:
    Account(int, double, Code<int> &);
    ...
};

Account::Account(int n, double b, Code<int> &c) :number(n), balance(b),code(c) {}
```

S.2 (Operator overloading&Exception Handling, 25p.) DİKKAT: Bu soruda, soru S.1’de yapılanlar dikkate alınmayacaktır.

(i) [5p] Code sınıfının bir üyesi olarak, (==) operator fonksiyonu yazınız. Bu operatör, compare fonksiyonu ile benzer işlem yürütecektir. (Örneğin, Code c1(1), c2(2); c1==c2 işlemi false döndürecektir.)

Cevap:

```
bool operator==(const Code& c) {  
    return (code == c.code);  
}
```

(iii) [5p] Account sınıfı için global bir fonksiyon olarak, (--=) operator fonksiyonu yazınız. withdraw fonksiyonu ile benzer çalışacaktır. (Örneğin, Account a1(10,2,Code(1)); a1--=3 işlemi sonrası balance değeri 7 olacaktır.)

Cevap:

```
class Account {  
    friend double operator--=(Account &, double );  
    ...  
  
double operator--=(Account &a, double amount) {  
    if ((a.balance - amount) >= 0) {  
        a.balance -= amount;  
        return a.balance;  
    }  
    else  
        return a.balance;  
}
```

iii) [7p] Account sınıfının constructor fonksiyonunda bir istisnanın (exception handling) dikkate alınması beklenmektedir. balance üye verisi, negatif değer alamamalıdır. Eğer, parametre olarak verilen balance değeri negatif ise throw ile string "negative balance" gönderilmelidir. Buna uygun şekilde, Account sınıfını constructor fonksiyonunu yeniden yazınız.

Cevap:

```
Account::Account(int n, double b, Code &c)  
    :number(n), balance(b), code(c) {  
    if((balance<0))  
        throw (string)"negative balance";  
    else  
        balance=b;  
}
```

iv) [8p] Basit bir main fonksiyon yazınız. Bu fonksiyonda, bir Account nesnesi yaratılacaktır. İstisnai duruma karşı eylem try ve catch ile gerçekleştirilecektir. Eylem, yakalanan string değerın ekrana yazdırılması olacaktır.

Cevap:

```
int main() {  
    try{  
        Account account(5,-5,Code(5));  
    }  
    catch(string s){ cout<<s<<endl; }  
    return 0;  
}
```

S.3 (Inheritance&polymorphism, 55p.) DİKKAT: Bu soruda, S.1’de yapılanlar dikkate alınmayacaktır. Aşağıdaki her şık önceki şıklarda yapılanları kapsamaktadır.

(a) [4p] Account sınıfını soyut (abstract) yapmak istiyoruz. Bunun için ne yapmak gerekir? Açıklayınız. (Standart C++ programlama dilinde olacak, derleyiciye özel yöntemler kabul edilmez.)

Cevap:

Üye fonksiyonlardan birisi, saf sanal fonksiyon (pure virtual function) olarak tanımlanır.

(b) [5p] Polymorphic (Çokyüzlü) kullanım için, Account sınıfından miras (inheritance) ile türetilen sınıflarda mutlaka withdraw() fonksiyonlarının bulunması ve kodlanması istenmektedir. Bunu sağlamak için, Account sınıfının kodlarında nasıl bir değişiklik yapmak gerekir. Değiştirilen kod satırlarını aşağıda yazınız.

Cevap:

Account.h

```
virtual double withdraw(double)=0;
```

Account.cpp

```
double Account::withdraw(double amount){...}
```

(c) [16p] Account sınıfından türetilen iki sınıf kodlayınız. Bunlardan birisi CAccount (checking account, vadesiz hesap) sınıfıdır. Taban sınıftakilere ek olarak, cBalance (ek hesap) ve const credit bilgisini taşımaktadır. cBalance değeri başlangıçta, credit değerine eşittir. Üye fonksiyonlar aşağıdaki gibi işletilmektedir:

- withdraw(double): Parametre olarak verilen miktar, balance (bakiye) den düşmekte, eğer bakiye sıfırlanmış ise, ek hesaptan düşmektedir. Ancak, bu işlemin gerçekleşebilmesi için, para çekimi sonrası bakiye ve ek hesabın negatif olmaması gerekir. Ek hesap en fazla credit değeri kadar olabilir.

Diğer sınıf, SAccount (savings account, vadeli hesap) sınıfıdır. Taban sınıftakilere ek olarak, sBalance ve interest bilgisi taşımaktadır. interest değeri 0 ile 1 arasındadır. Başlangıçta, sBalance sıfırdır. Üye fonksiyonlar aşağıdaki gibi işletilmektedir:

- withdraw(double): Parametre olarak verilen miktar, balance (bakiye) den düşmektedir. Ancak, bu işlemin gerçekleşebilmesi için, para çekimi sonrası bakiyenin negatif olmaması gerekir.

Cevap:

CheckingAccount.h <pre>class CAccount :public Account { double cBalance; const double credit; public: CAccount(int, double, Code &, double); double withdraw(double); };</pre>	SavingsAccount.h <pre>class SAccount :public Account { double sBalance; double interest; public: SAccount(int, double, Code &, double); double withdraw(double); };</pre>
CheckingAccount.cpp <pre>CAccount::CAccount(int n, double b, Code &c, double _credit) :Account(n,b,c),credit(_credit){ cBalance = credit; } double CAccount::withdraw(double amount) { double tmp; if (amount <= (getBalance() + cBalance)) { if (getBalance() >= amount) (*this) -= amount; else { tmp = amount - getBalance(); cBalance -= tmp; (*this) -= getBalance(); } return amount; } return 0; }</pre>	SavingsAccount.cpp <pre>SAccount::SAccount(int n, double b, Code &c, double _int) :Account(n, b, c), interest(_int),sBalance(0) {} double SAccount::withdraw(double amount) { if (amount <= getBalance()) { (*this) -= amount; return amount; } return 0; }</pre>

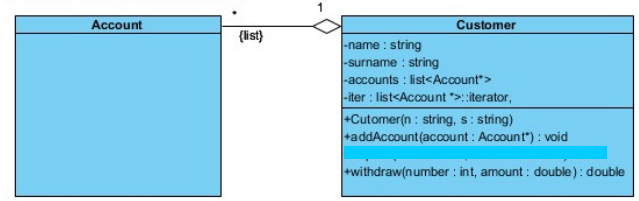
(d) [18p] Banka müşterileri, çok sayıda vadeli ve vadesiz hesaba sahip olabilirler. Bir Customer (müşteri) sınıfı tasarlanacaktır. Müşterinin adına açılan tüm hesapların nesne adresleri list (std, standard template library, standart şablon kütüphanesi içinde sunulan) taşıyıcısı içerisinde tutması istenmektedir. Buna göre Customer sınıfının kodlamasını yazınız. (addAccount, parametre olarak verilen hesap adreslerini listeye eklemektedir; withdraw, ilk parametre ile verilen numaraya sahip hesap listede aranır, bulunursa ikinci parametre ile verilen para çekilir, bulunmazsa çekme gerçekleşmez.)

Cevap:

```
class Customer {
    string name;
    string surname;
    list<Account*> accounts;
    list<Account*>::iterator iter;
public:
    Customer(string n="", string s=""):name(n),surname(s){}
    void addAccount(Account *account) {
        accounts.push_back(account);
    }

    double withdraw(int number, double amount) {
        for (iter = accounts.begin(); iter != accounts.end(); iter++) {
            if ((*iter)->getNumber() == number) {
                (*iter)->withdraw(amount);
                return amount;
            }
        }
        return 0;
    }
};
```

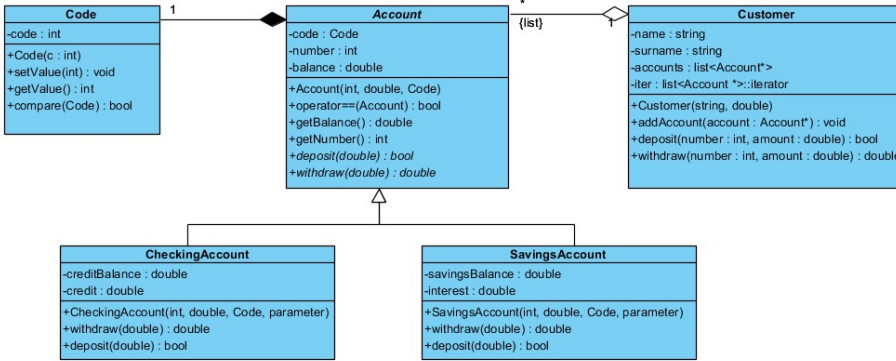
Visual Paradigm Standard(Eskişehir Osmangazi University)



(e) [12p] Yazılımda bulunan ve şu ana kadar bahsedilen tüm sınıfları ve ilişkilerini gösteren UML sınıf diyagramını çiziniz.

Cevap:

Visual Paradigm Standard(Eskişehir Osmangazi University)



Başarılar...